

Induction, Free algebra, Pattern Matching

CC-BY 2025 James B. Wilson

1. Tree Operators

You are shown the following lines of code in a programming language called Swift. You suspect it is an inductive type. If so, what are its operators?
[Note: you don't need to know anything about Swift to do this exercise.]

```
indirect enum BinaryTree<A> {
    case leaf(A)
    case node(BinaryTree<A>, BinaryTree<A>)
}
```

Solution.

I will call the algebra $\mathbb{BT}$. I see `leaf` takes in data of type A but no tree types so it is constant for each term $a : A$ so that gives us these constants.

$$l_a : \mathbb{BT}^0 \rightarrow \mathbb{BT} \text{ one for each } a : A$$

I also see `node` depends on two trees so that will be an operator with 2 inputs of type $\mathbb{BT}$ hence

$$n : \mathbb{BT}^2 \rightarrow \mathbb{BT}$$

2. A task algebra

You need to track complex tasks (i.e. ones built from simpler tasks) that come in these categories.

- Parallel: here pairs (t_1, t_2) are tasks that can be run in parallel.
- Sequential: here pairs (t_1, t_2) of tasks can only be run sequentially t_1 before t_2 .
- Identity: a task that keeps all the data the same.
- Null: a task that stops any future work.
- Read: a task that reads one packet of data.
- Write: a task that writes one packet of data.
- Transform: a task that transforms data.

If T is the type for all tasks, write the algebra operators for creating complex tasks.

Solution.. I will use $\mathbb{T}$ for "tasks". For parallel I am told two tasks are needed, so that corresponds to a valence 2 operator. I will shorten the name to p . (After the computer doesn't read in to the meaning of the English word "parallel" so I shouldn't get too hung up on the name either lest I introduce thinking that isn't really part of the code).

$$\text{parallel} : \mathbb{T}^2 \rightarrow \mathbb{T}$$

Sequential is also based on pairs, it does something different from parallel but still it is a binary, i.e. valence 2, operator.

$$\text{sequential} : \mathbb{T}^2 \rightarrow \mathbb{T}$$

Identity? That starts from nothing, i.e. $() \in \mathbb{T}^0 = \{()\}$

$$\text{identity} : \mathbb{T}^0 \rightarrow \mathbb{T}$$

Null is a constant task as well.

$$\text{null} : \mathbb{T}^0 \rightarrow \mathbb{T}$$

Reading? Writing? Transform? These both depend on some data, but again like null and identity are just constant tasks.

$$\begin{aligned}\text{read} &: \mathbb{T}^0 \rightarrow \mathbb{T} \\ \text{write} &: \mathbb{T}^0 \rightarrow \mathbb{T} \\ \text{transform} &: \mathbb{T}^0 \rightarrow \mathbb{T}\end{aligned}$$

3. A polynomial algebra

You need an algebra that has a binary addition $_ + _$, a binary multiplication $_ \cdot _$, a constant 1 , a constant 0 , a variable x , a variable y , or a variable z . Write the operators of the algebra.

Solution. I am gonna call this algebra $\mathbb{P}$ for "polynomial"

$$\begin{aligned}+ &: \mathbb{P}^2 \rightarrow \mathbb{P} \\ \cdot &: \mathbb{P}^2 \rightarrow \mathbb{P} \\ 1 &: \mathbb{P}^0 \rightarrow \mathbb{P} \\ 0 &: \mathbb{P}^0 \rightarrow \mathbb{P} \\ x &: \mathbb{P}^0 \rightarrow \mathbb{P} \\ y &: \mathbb{P}^0 \rightarrow \mathbb{P} \\ z &: \mathbb{P}^0 \rightarrow \mathbb{P}\end{aligned}$$

4. Polynomial-task algebra

Suppose that $a : \{x, y, z\} \rightarrow \{\text{Read, Write, Transform}\}$ is a function. Define the associated evaluation homomorphism

$$\text{eval}_a(\phi(x, y, z)) := \phi(\text{Read, Write, Transform})$$

Write a pattern matching pseudo-code for this evaluation in a programming language style of your liking. Finally what does $x \cdot (y + 1) \cdot x \cdot z$ translate into? [Hint: it may help you to first parse the formula into a tree to see the steps.]

When done correctly you should see that the algebra of 2 is exactly the algebra of 3. What it means is that every polynomial in x, y, z with positive integers coefficients is actually encoding the layout of a distributed computation. You can even begin to organize your program by moving around the polynomial terms and using your algebraic experience.

Solution.

$$\text{eval}_a(\phi(x, y, z)) = \begin{cases} \text{parallel}(\lambda(x, y, z), \rho(x, y, z)) & \phi(x, y, z) = \lambda(x, y, z) + \rho(x, y, z) \\ \text{sequential}(\lambda(x, y, z), \rho(x, y, z)) & \phi(x, y, z) = \lambda(x, y, z) \cdot \rho(x, y, z) \\ \text{identity} & \phi(x, y, z) = 1 \\ \text{null} & \phi(x, y, z) = 0 \\ \text{read} & \phi(x, y, z) = x \\ \text{write} & \phi(x, y, z) = y \\ \text{transform} & \phi(x, y, z) = z \end{cases}$$